Informatics Institute of Technology (IIT) Sri Lanka, Affiliated with The University of Westminster, UK Module

Client-Server Architectures

5COSC022W

Siddiha Rimzan
UOW Number- w2120139
IIT number- 20240856

Part 1: Service Architecture & Setup

Q1: In your report, explain the default lifecycle of a JAX-RS Resource class. Is anew instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures $(maps/lists)$ to prevent data loss or race conditions.

By default, JAX-RS creates a brand new instance of a resource class for every incoming HTTP request. This is known as per-request scope. The JAX-RS runtime instantiates the class, handles the request, and then discards the object. This means that instance variables inside a resource class are not shared between requests.

This architectural decision has a direct impact on how shared in-memory data must be managed. Since each request gets its own resource object, you cannot store shared state such as a list of rooms or sensors inside the resource class itself. If you did, data would be lost after every request.

To solve this, the Singleton pattern is used via a dedicated DataStore class. This class holds a single static instance that persists throughout the lifetime of the application, shared across all requests. Additionally, ConcurrentHashMap is used instead of a regular HashMap to ensure thread safety. ConcurrentHashMap allows multiple threads to read and write simultaneously without causing race conditions or data corruption, which is critical when multiple requests arrive at the same time.

Q2: Why is the provision of ”Hypermedia” (links and navigation within responses)considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

HATEOAS stands for Hypermedia as the Engine of Application State. It is a constraint of REST architecture where API responses include hyperlinks that guide the client to related actions and resources. Rather than the client needing to know all possible URLs in advance, the API itself tells the client what it can do next.

For example, when a client fetches a room resource, the response could include links such as a link to view its sensors, a link to delete the room, or a link to add a new sensor. This makes the API self-documenting and self-navigating.

This benefits client developers significantly compared to static documentation because the client does not need to hardcode URLs or memorise the API structure. If the API changes its URL structure, clients that follow hypermedia links automatically adapt without breaking. It reduces the coupling between the client and server, making the system far more maintainable and scalable in the long run.

Part 2: Room Management

Q3: When returning a list of rooms, what are the implications of returning onlyIDs versus returning the full room objects? Consider network bandwidth and client side processing.

When returning a list of rooms, there are two main design options: returning only the room IDs, or returning the full room objects including all fields such as name, capacity, and sensor IDs.

Returning only IDs is lightweight and reduces network bandwidth significantly, especially when there are thousands of rooms. However, it forces the client to make additional requests to retrieve the details of each room individually, which increases the number of round trips and places more processing load on the client side.

Returning full room objects provides all the information in a single response, reducing the number of API calls the client needs to make. This is more efficient for use cases where the client needs to display a list of rooms with their details. The trade-off is a larger response payload, which may be a concern over slow networks or when dealing with very large datasets. In practice, the full object approach is preferred for usability, and pagination should be implemented to manage large datasets.

Q4:Is the DELETE operation idempotent in your implementation? Provide a detailedjustification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times.

Yes, the DELETE operation is idempotent in this implementation. Idempotency means that performing the same operation multiple times produces the same result as performing it once.

In this API, if a client sends a DELETE request for a room that exists, the room is removed and a 200 OK response is returned. If the same DELETE request is sent again for the same room ID, the room no longer exists, so the server returns a 404 Not Found response. The state of the server remains the same after the second request as it was after the first, which means the operation is idempotent. The room is still gone regardless of how many times the DELETE is called. The response code may differ between the first and subsequent calls, but the server state does not change, which satisfies the definition of idempotency.

Part 3: Sensor Operations & Linking

Q5:We explicitly use the @Consumes (MediaType.APPLICATION_JSON) annotation onthe POST method. Explain the technical consequences if a client attempts to send data ina different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

The @Consumes(MediaType.APPLICATION_JSON) annotation tells the JAX-RS runtime that the POST endpoint only accepts requests with a Content-Type header of application/json. If a client sends data with a different format such as text/plain or application/xml, JAX-RS will automatically reject the request before it even reaches the method body.

The runtime will return an HTTP 415 Unsupported Media Type response. This means the server is explicitly telling the client that the format it sent is not supported. The method will not be invoked at all, and no partial processing will occur. This is an important safety feature because it prevents malformed or unexpected data from entering the system, and it enforces a clear contract between the client and the server regarding the expected data format.

Q6: You implemented this filtering using @QueryParam. Contrast this with an alternativedesign where the type is part of the URL path (e.g., /api/vl/sensors/type/CO2). Whyis the query parameter approach generally considered superior for filtering and searching collections?

Using @QueryParam for filtering, such as GET /api/v1/sensors?type=CO2, is considered superior to embedding the filter in the URL path, such as GET /api/v1/sensors/type/CO2, for several important reasons.

Query parameters are optional by nature. The same endpoint can serve both the full list and a filtered list depending on whether the parameter is provided. This keeps the API clean and avoids having multiple separate endpoints for different filter combinations.

Path parameters are typically used to identify a specific resource by its unique identifier, such as /sensors/TEMP-001. Using path segments for filtering blurs this distinction and makes the URL structure ambiguous. It can also cause conflicts with other path-based routes.

Query parameters also support multiple filters easily, such as ?type=CO2&status=ACTIVE, without requiring changes to the URL structure. This makes the API far more flexible and scalable for future filtering requirements.

Part 4: Deep Nesting with Sub - Resources

Q7: Discuss the architectural benefits of the Sub-Resource Locator pattern. Howdoes delegating logic to separate classes help manage complexity in large APIs comparedto defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive controller class?

The Sub-Resource Locator pattern allows a resource class to delegate handling of nested paths to a separate dedicated class. In this API, the SensorResource class delegates the /sensors/{sensorId}/readings path to a separate SensorReadingResource class rather than defining all reading-related methods inside SensorResource.

This approach provides significant architectural benefits in terms of managing complexity. Each class has a single clear responsibility, which follows the Single Responsibility Principle. SensorResource focuses on sensor-level operations, while SensorReadingResource focuses entirely on reading-level operations. This separation makes the code far easier to read, maintain, and test independently.

In large APIs with many nested resources, defining every endpoint in one massive controller class quickly becomes unmanageable. The class grows too large, making it difficult to locate specific logic and increasing the risk of introducing bugs when making changes. By delegating to separate classes, each resource remains concise and focused. New developers can understand the structure at a glance, and changes to reading logic do not risk affecting sensor logic.

Part 5: Advanced Error Handling & Exception Mapping

Q8: Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

When a client sends a POST request to create a new sensor and provides a roomId that does not exist in the system, the issue is not that the URL or endpoint cannot be found. The request URL is perfectly valid, and the endpoint exists and is reachable. The problem lies within the content of the request body itself, specifically a reference to a resource that does not exist.

A 404 Not Found response is semantically incorrect in this scenario because 404 implies that the requested URL or endpoint does not exist, which is misleading. The endpoint is found and processing began successfully.

HTTP 422 Unprocessable Entity is more semantically accurate because it communicates that the server understood the request, received valid JSON, but could not process it due to a logical or semantic error within the payload. In this case, the payload references a roomId that does not exist, making the entity semantically invalid even though it is syntactically correct JSON. This gives the client a much clearer signal about what went wrong and how to fix it.

Q9: From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

Exposing internal Java stack traces to external API consumers poses serious cybersecurity risks. A stack trace reveals detailed internal information about the application that attackers can exploit in several ways.

First, a stack trace exposes the internal package and class names, method names, and line numbers of the source code. This gives an attacker a precise map of the application's internal architecture, making it far easier to identify entry points and vulnerabilities.

Second, stack traces often reveal the technology stack being used, such as the version of Jersey, Java, or specific libraries. Attackers can cross-reference known vulnerabilities for those specific versions and craft targeted exploits.

Third, error messages within stack traces sometimes include sensitive data such as database connection strings, file paths, configuration details, or even partial data values. This information can be used for SQL injection, path traversal attacks, or other targeted exploits.

By implementing a global exception mapper that returns a generic 500 Internal Server Error message without any stack trace details, the API ensures that internal implementation details are never leaked to external consumers, significantly reducing the attack surface.